

Instalar el SDK de Anthropic y las claves de API

Claude Certified Developer, Foundation (CCDV-F) · Masterclass UST

ES Sesión 7 de 12 · Notas en español (es-ES)

La sesión 7 (17 de agosto de 2026) pasó de los productos con interfaz a **llamar a Claude desde código**. El laboratorio de Vijay usó notebooks de Python: instalar el SDK de Anthropic, cargar un secreto desde `.env`, crear un cliente y llamar a `messages.create`. El examen cuida la terminología (SDK frente a REST), dónde vive la clave y qué método invocáis — no el orden de las celdas del notebook.

Dos caminos hacia la misma Messages API

Anthropic expone Claude por HTTP. Podéis hacer POST al endpoint REST de Messages vosotros (URL, cabeceras, cuerpo JSON, clave). El **SDK de Anthropic** es un envoltorio por lenguaje sobre esa superficie REST. En Python es el paquete que instaláis; en Java sería un artefacto SDK. Una pregunta del grupo lo dejó claro: en Python se dice «librería»; el examen dice **SDK (software development kit)**. Misma idea: ayudas tipadas, reintentos, streaming y un objeto cliente para no montar cada petición a mano.

Camino	Qué controláis	Cuándo usarlo
REST	Cliente HTTP, autenticación, esquema JSON, errores	Stacks sin Python, pasarelas, o cuando hay que ver el cable en bruto
SDK	Instalar el paquete, crear el cliente, llamar métodos	Por defecto en clase; apps Python/TypeScript
Agent SDK (más adelante)	Orquestación sobre mensajes y tools	Agentes — no es el cliente de este capítulo

EL SDK DE ANTHROPIC NO ES EL AGENT SDK

Vijay distinguió el **SDK de Anthropic** (conversaciones / Messages API) del **Agent SDK** (orquestación). Tools, MCP y agentes vuelven en módulos posteriores. Las preguntas Foundation de esta semana giran en torno a `client.messages.create`, no LangChain.

Instalación, entorno y el objeto cliente

El patrón del laboratorio: instalar dependencias en el notebook (a menudo con `%%capture` de Jupyter para no inundar la celda), cargar variables de entorno, instanciar `anthropic.Anthropic()` (o el constructor vigente). Ese cliente agrupa recursos. Bajo **messages** vimos al menos `create` (una llamada, sin streaming por defecto), `stream` y **batches**. Se accede con notación de punto: `client.messages.create(...)`.

- Las primeras celdas de los notebooks de Vijay son el andamiaje: `SDK`, `load_dotenv()`, cliente, id de modelo por defecto.

- El repositorio GitHub de la clase trae notebooks; vosotros creáis el `.env` con la clave. No lo subáis al control de versiones.
- La primera llamada en frío puede tardar: red y arranque del modelo, no un fallo de instalación.

Claves de API: consola, `.env` y lo que inyecta el SDK

La autenticación es una **clave secreta de API** emitida en la plataforma / consola de Anthropic, no la suscripción Claude Pro ni Claude Code. Vijay la guardó en `.env` y la cargó con `dotenv` para que el proceso tuviera la variable. El SDK la adjunta en cada petición. Podéis pasar la clave al constructor; no la pegéis en cada celda. Eso es precisamente un kit de desarrollo: secretos fuera del código fuente.

Preguntas de aula que salen en examen

- No hace falta pasar la clave como parámetro de `create` si el cliente ya cargó `ANTHROPIC_API_KEY` (nombre según la documentación actual del SDK).
- Claude Pro / Claude Code no son el mismo modelo de facturación que los créditos de API.
- En cuentas de empresa el formador puede no poder compartir la generación de claves. Aprended el flujo en la documentación oficial.
- Las cuentas nuevas se describieron con un crédito inicial pequeño (Vijay habló del orden de 5 \$); las cantidades caducan: no las memoricéis como dogma de examen.

CONFIDENCIALIDAD

No pongáis secretos de clientes UST en una clave personal, un notebook compartido o un GitHub público. Rotad la clave si se pegó en el chat o se subió al repositorio.

La llamada `create`: `modelo`, `max_tokens`, `messages`

Una invocación mínima necesita tres ideas aunque el SDK tenga valores por defecto: **qué modelo** (cadena de id), **max_tokens** (techo de tokens *generados* — en clase 1024 como valor de trabajo) y **messages** (lista de objetos rol/contenido). El contenido no es «un prompt suelto»: cada elemento tiene **role** (user, luego assistant) y **content** (texto en el demo Hello Claude). En una aplicación, el contenido de usuario es la entrada real sustituida en esa estructura.

El valor de retorno es un objeto **Message** (como un diccionario en Python). Lo habitual es recorrer `message.content`, tomar el primer bloque y leer `.text` en un bloque de texto. Imprimir el objeto entero sirve en clase porque también ves `id`, `model`, `role` (assistant), `stop_reason`, `usage` y más adelante `container` (caché / enrutado — módulo 09).

Aspecto de un Message correcto

- **id** — identificador único del mensaje en la API.

- **content** — lista de bloques. Las respuestas de texto llegan como bloque **text** con propiedad `text`. Los arrays de citas aparecen al activar citations (módulo 09 / RAG).
- **role** — assistant para la salida del modelo.
- **stop_reason / stop_details** — por qué paró la generación. Si no tocáis el techo de salida, suele haber un fin de turno limpio. Si `max_tokens` corta la respuesta, los detalles explican el corte brusco.
- **usage** — recuento de tokens de entrada frente a salida (y luego caché) de esa petición.

Cuerpo REST frente a parámetros del SDK

La plataforma sigue siendo REST: cada llamada del SDK es URL de Anthropic + cuerpo JSON. El cuerpo incluye `model`, `messages`, `system` (si hay), `temperature`, `max_tokens`, `stream`, `tools`, `thinking`, `cache control`, etc. **La API es sin estado.** Repetir el `system` prompt en cada petición no es un error; así funcionan las APIs HTTP. La caché de prompts (módulo 09) es la vía soportada para no volver a pagar un prefijo enorme y estable.

DOCUMENTACIÓN POR ENCIMA DE LA UI DEL NOTEBOOK

Nombres de paquetes, firmas del constructor y magia de Jupyter cambian. Priman la **Messages API** y el README actual del SDK de Python si una captura de clase discrepa.

Lista de estudio

1. Nombrad los dos caminos: REST y SDK; el SDK envuelve REST.
2. Decid dónde se crea la clave (consola de API) y cómo la cargó la clase (`.env` + `dotenv`).
3. Listad métodos de `messages` citados: `create`, `stream`, `batches`.
4. Nombrad las tres entradas centrales de `create`: `model`, `max_tokens`, `messages`.
5. Extraed el texto del asistente de `content[0].text` (o equivalente) sin volcar el objeto entero en producción.

Documentación oficial

• Messages API

<https://docs.claude.com/en/api/messages>

Campos canónicos de petición y respuesta de `create`.

• Visión general de la API

<https://docs.claude.com/en/api/overview>

Autenticación, clientes y organización de la API HTTP.

• Workspaces

<https://docs.claude.com/en/docs/build-with-claude/workspaces>

Dónde se agrupan claves y proyectos en la consola.

• Límites de tasa

<https://docs.claude.com/en/api/rate-limits>

Por qué un primer demo puede fallar con una clave válida.

- **Dónde está la documentación de la API**

<https://support.claude.com/en/articles/8114490-where-can-i-find-your-api-documentation>

Artículo de soporte que apunta a docs.claude.com.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic's current docs over session anecdotes. Not official exam material.